

Tiny Stories Small Language Model

Objective:

I had constructed a small language model with just 2 millions parameters, and which can produce coherent text or generate small stories.

Purpose:

Building language model. So that ultimately it produces stories which look like the one in the dataset.

About Dataset:

Our dataset has only stories for the 2 to 4 years old kids . We are going to use this dataset to teach our language model about the english language. So that ultimately it produces stories.

Tiny Stories is a synthetic dataset of short stories that only contain words that a typical 2 to 4-year-olds kids usually understand, generated by GPT-3.5 and GPT-4. We can get it from Hugging Face.

No. of rows: 2,141,709

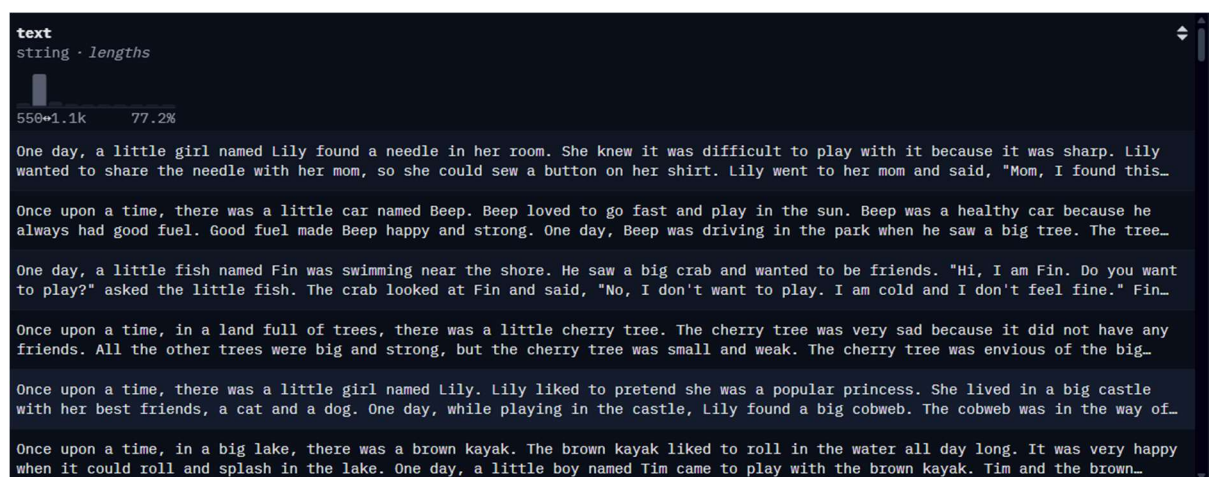
Split:

Train: 2.12 million (rows)

Validation: 22K (rows)

Datatype: String

Link for dataset: <https://huggingface.co/datasets/roneneldan/TinyStories>



The screenshot shows a text editor with a dark background. At the top left, it says 'text' and 'string · lengths'. Below this, there is a progress bar and the text '558*1.1k 77.2%'. The main content of the editor is a list of short stories, each starting with 'Once upon a time, ...'. The stories are about a girl named Lily, a car named Beep, a fish named Fin, a cherry tree, and a girl named Lily. The text is truncated at the end of each story with an ellipsis.

text
string · lengths

558*1.1k 77.2%

Once upon a time, a little girl named Lily found a needle in her room. She knew it was difficult to play with it because it was sharp. Lily wanted to share the needle with her mom, so she could sew a button on her shirt. Lily went to her mom and said, "Mom, I found this_

Once upon a time, there was a little car named Beep. Beep loved to go fast and play in the sun. Beep was a healthy car because he always had good fuel. Good fuel made Beep happy and strong. One day, Beep was driving in the park when he saw a big tree. The tree_

Once upon a time, a little fish named Fin was swimming near the shore. He saw a big crab and wanted to be friends. "Hi, I am Fin. Do you want to play?" asked the little fish. The crab looked at Fin and said, "No, I don't want to play. I am cold and I don't feel fine." Fin_

Once upon a time, in a land full of trees, there was a little cherry tree. The cherry tree was very sad because it did not have any friends. All the other trees were big and strong, but the cherry tree was small and weak. The cherry tree was envious of the big_

Once upon a time, there was a little girl named Lily. Lily liked to pretend she was a popular princess. She lived in a big castle with her best friends, a cat and a dog. One day, while playing in the castle, Lily found a big cobweb. The cobweb was in the way of_

Once upon a time, in a big lake, there was a brown kayak. The brown kayak liked to roll in the water all day long. It was very happy when it could roll and splash in the lake. One day, a little boy named Tim came to play with the brown kayak. Tim and the brown_

Introduction:

Let us build a Small Language Model (SLM) from scratch. We will try to keep the parameter size to 2.1 million.

Our goal is to generate creative and coherent text based on the input data.

Methodology:

Step 1: Import the Dataset

Tiny Stories is a synthetic dataset of short stories that only contain words that a typical 1 to 4-year-olds kids usually understand, generated by GPT-3.5 and GPT-4. We can get it from Hugging Face.

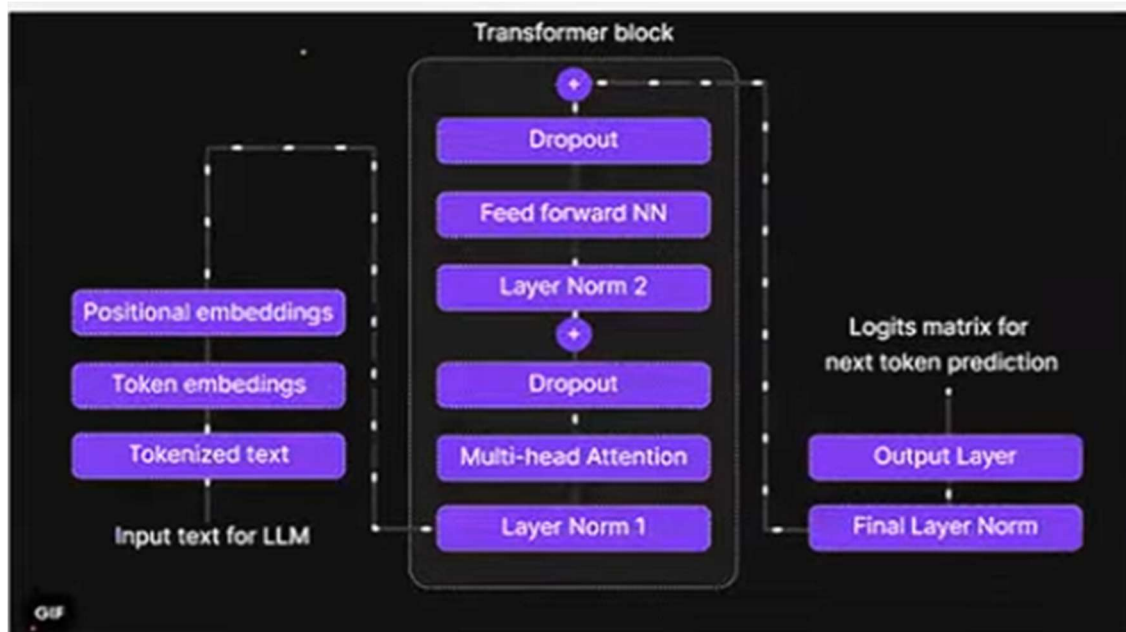
Step 2: Tokenize the Dataset

In this step, we will do the following:

- (1) Tokenize the dataset into tokenIDs.
- (2) Create a file called "train.bin" and "validation.bin" where we will store the tokenIDs from the entire dataset.
- (3) We make sure the tokenIDs are stored on a disk, rather than on the RAM for efficient computations.

Step 3: Create Input-Output batches for the dataset

Step 4: Define the SLM Model Architecture



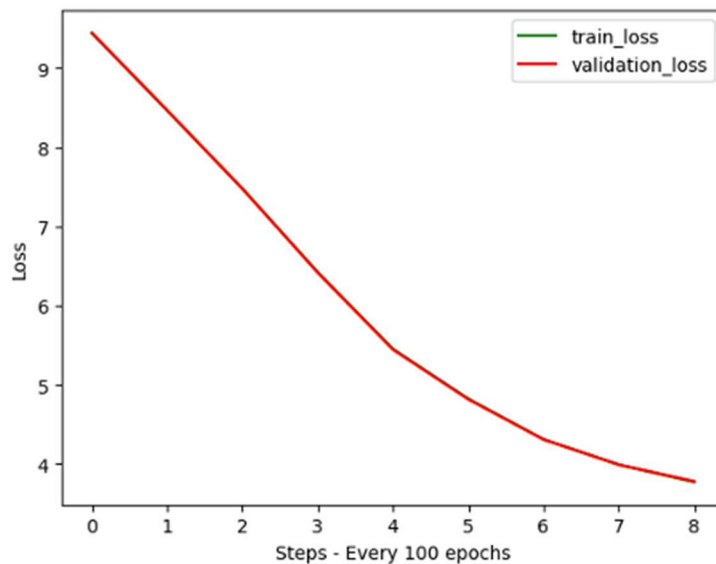
Step 5: Define the loss function

Step 6: Define SLM Training Configuration Part 1

Step 7: Define SLM Training Configuration Part 2

Step 8: Pre-train the SLM

Step 9: Plot the SLM Loss Function



Step 10: Run SLM Inference on our trained model

```
sentence = "Once upon a time there was a rabbit."  
context = (torch.tensor(enc.encode_ordinary(sentence)).unsqueeze(dim = 0))  
y = model.generate(context, 200)  
print(enc.decode(y.squeeze().tolist()))
```

Once upon a time there was a rabbit. And, there was a girl named tricks. Once upon it with it to banana little boy who was st
Her mom made higher as she saw a very whclude. Once upon a time, "You fight that be better. there was a you like to make hers

One under the slide and the park.
As he taken its fun day, came to join the frog. She decided to "
Amy took a whistle andy am so to tell Mom still sad and turned

```
sentence = "A little boy went to the jungle"  
context = (torch.tensor(enc.encode_ordinary(sentence)).unsqueeze(dim = 0))  
y = model.generate(context, 200)  
print(enc.decode(y.squeeze().tolist()))
```

A little boy went to the jungle high and wanted, when her hand. Then she started to eat with a big smile. Suddenly, away and
As they tower with a big very unhappy. After pretty said I decided to be a small water cold to be Her mom and had sure.

But she went excited to delicate one day on the woods before she run across the park with her happy a made anard, but she pl

One had a tall band happily returned to each hat and someone felt two but he thought it lazy truck was a leash and better. C

Introduction

Let us build a Small Language Model (SLM) from scratch. We will try to keep the parameter size to 2.1 million.

Our goal is to generate creative and coherent text based on the input data.

Step 1: Import the Dataset

TinyStories is a synthetic dataset of short stories that only contain words that a typical 1 to 4-year-olds kids usually understand, generated by GPT-3.5 and GPT-4. We can get it from HuggingFace.

```
!pip install datasets
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.24.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (2025.3.0)
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.4.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!<4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (4.0.0a1)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (1.2.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (0.23.0)
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (1.0.0)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (0.10.0)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (4.10.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2) (2025.1.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (25.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (6.0.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1) (1.17.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0) (4.7.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*) (0.16.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2) (1.17.0)
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim) (0.24.0)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0) (8.2.1)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0) (13.9.0)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0) (0.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0) (0.1.2)
```

```
from datasets import load_dataset
```

```
ds = load_dataset("roneneldan/TinyStories")
```

Step 2: Tokenize the Dataset

In this step, we will do the following:

- (1) Tokenize the dataset into tokenIDs.
- (2) Create a file called "train.bin" and "validation.bin" where we will store the tokenIDs from the entire dataset.
- (3) We make sure the tokenIDs are stored on a disk, rather than on the RAM for efficient computations.

```
!pip install tiktoken
import tiktoken
import os
import numpy as np
from tqdm.auto import tqdm

enc = tiktoken.get_encoding("gpt2")
```

```
# Some functions from https://github.com/karpathy/nanoGPT/blob/master/data/openwebtext/prepare.py

def process(example):
    ids = enc.encode_ordinary(example['text']) # encode_ordinary ignores any special tokens
    out = {'ids': ids, 'len': len(ids)}
    return out

if not os.path.exists("train.bin"):
    tokenized = ds.map(
        process,
        remove_columns=['text'],
        desc="tokenizing the splits",
        num_proc=8,
    )
    # concatenate all the ids in each dataset into one large file we can use for training
    for split, dset in tokenized.items():
        arr_len = np.sum(dset['len'], dtype=np.uint64)
        filename = f'{split}.bin'
        dtype = np.uint16 # (can do since enc.max_token_value == 50256 is < 2**16)
        arr = np.memmap(filename, dtype=dtype, mode='w+', shape=(arr_len,))
        total_batches = 1024

        idx = 0
        for batch_idx in tqdm(range(total_batches), desc=f'writing {filename}'):
            # Batch together samples for faster write
            batch = dset.shard(num_shards=total_batches, index=batch_idx, contiguous=True).with_format('numpy')
            arr_batch = np.concatenate(batch['ids'])
            # Write into mmap
            arr[idx : idx + len(arr_batch)] = arr_batch
            idx += len(arr_batch)
        arr.flush()
```

Requirement already satisfied: tiktoken in /usr/local/lib/python3.12/dist-packages (0.12.0)
 Requirement already satisfied: regex>=2022.1.18 in /usr/local/lib/python3.12/dist-packages (from tiktoken) (2025.11.3)
 Requirement already satisfied: requests>=2.26.0 in /usr/local/lib/python3.12/dist-packages (from tiktoken) (2.32.4)
 Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.26.0->tiktoken) (3.3.0)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.26.0->tiktoken) (3.10.1)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.26.0->tiktoken) (2.2.3)
 Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.26.0->tiktoken) (2025.11.12)

Step 3: Create Input-Output batches for the dataset

```
# Some functions from https://github.com/karpathy/nanoGPT/blob/master/train.py with slight modifications
#block size = context window
def get_batch(split):
    # We recreate np.memmap every batch to avoid a memory leak, as per
    # https://stackoverflow.com/questions/45132940/numpy-memmap-memory-usage-want-to-iterate-once/61472122#61472122
    if split == 'train':
        data = np.memmap('train.bin', dtype=np.uint16, mode='r')
    else:
        data = np.memmap('validation.bin', dtype=np.uint16, mode='r')
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([torch.from_numpy((data[i:i+block_size]).astype(np.int64))) for i in ix])
    y = torch.stack([torch.from_numpy((data[i+1:i+1+block_size]).astype(np.int64))) for i in ix])
    if device_type == 'cuda':
        # pin arrays x,y, which allows us to move them to GPU asynchronously (non_blocking=True)
        x, y = x.pin_memory().to(device, non_blocking=True), y.pin_memory().to(device, non_blocking=True)
    else:
        x, y = x.to(device), y.to(device)
    return x, y
```

Step 4: Define the SLM Model Architecture

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
from dataclasses import dataclass
import numpy as np
from tqdm.auto import tqdm
from contextlib import nullcontext
import os

class LayerNorm(nn.Module):
    def __init__(self, ndim, bias):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(ndim))
        self.bias = nn.Parameter(torch.zeros(ndim)) if bias else None
    def forward(self, x):
```

```

        return F.layer_norm(x, self.weight.shape, self.weight, self.bias, 1e-5)

class CausalSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        assert config.n_embd % config.n_head == 0
        self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd, bias=config.bias)
        self.c_proj = nn.Linear(config.n_embd, config.n_embd, bias=config.bias)
        self.attn_dropout = nn.Dropout(config.dropout)
        self.resid_dropout = nn.Dropout(config.dropout)
        self.n_head = config.n_head
        self.n_embd = config.n_embd
        self.flash = hasattr(F, 'scaled_dot_product_attention')
        if not self.flash:
            self.register_buffer("bias", torch.tril(torch.ones(config.block_size, config.block_size))
                                .view(1, 1, config.block_size, config.block_size))

    def forward(self, x):
        B, T, C = x.size()
        q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
        k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
        q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
        v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)

        if self.flash:
            y = F.scaled_dot_product_attention(q, k, v, attn_mask=None, dropout_p=self.attn_dropout.p if self.training else
            else:
                att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
                att = att.masked_fill(self.bias[:, :, :T, :T] == 0, float('-inf'))
                att = F.softmax(att, dim=-1)
                att = self.attn_dropout(att)
                y = att @ v

        y = y.transpose(1, 2).contiguous().view(B, T, C)
        y = self.resid_dropout(self.c_proj(y))
        return y

class MLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd, bias=config.bias)
        self.gelu = nn.GELU()
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd, bias=config.bias)
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x):
        return self.dropout(self.c_proj(self.gelu(self.c_fc(x))))

class Block(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.ln1 = LayerNorm(config.n_embd, config.bias)
        self.attn = CausalSelfAttention(config)
        self.ln2 = LayerNorm(config.n_embd, config.bias)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(self.ln1(x))
        x = x + self.mlp(self.ln2(x))
        return x

@dataclass
class GPTConfig:
    block_size: int
    vocab_size: int
    n_layer: int
    n_head: int
    n_embd: int
    dropout: float = 0.0
    bias: bool = True

class GPT(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.config = config
        self.transformer = nn.ModuleDict(dict(
            wte=nn.Embedding(config.vocab_size, config.n_embd),
            wpe=nn.Embedding(config.block_size, config.n_embd),
            drop=nn.Dropout(config.dropout),
            h=nn.ModuleList([Block(config) for _ in range(config.n_layer)]),
            ln_f=LayerNorm(config.n_embd, config.bias),
        ))
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
        self.transformer.wte.weight = self.lm_head.weight # weight tying

```

```

        self.apply(self._init_weights)
        for pn, p in self.named_parameters():
            if pn.endswith('c_proj.weight'):
                nn.init.normal_(p, mean=0.0, std=0.02 / math.sqrt(2 * config.n_layer))

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, idx, targets=None):
        device = idx.device
        b, t = idx.size()
        assert t <= self.config.block_size
        pos = torch.arange(0, t, dtype=torch.long, device=device)

        tok_emb = self.transformer.wte(idx)
        pos_emb = self.transformer.wpe(pos)
        x = self.transformer.drop(tok_emb + pos_emb)
        for block in self.transformer.h:
            x = block(x)
        x = self.transformer.ln_f(x)

        if targets is not None:
            logits = self.lm_head(x)
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1), ignore_index=-1)
            return logits, loss
        else:
            logits = self.lm_head(x[:, [-1], :])
            return logits, None

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None):
        """
        Generate tokens given a conditioning sequence.
        idx: Tensor of shape (B, T)
        """
        for _ in range(max_new_tokens):
            idx_cond = idx if idx.size(1) <= self.config.block_size else idx[:, -self.config.block_size:]
            logits, _ = self(idx_cond)
            logits = logits[:, -1, :] / temperature
            if top_k is not None:
                v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
                logits[logits < v[:, [-1]]] = -float('Inf')
            probs = F.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
            idx = torch.cat((idx, idx_next), dim=1)
        return idx

```

```

config = GPTConfig(
    vocab_size=50257,      # use the tokenizer's vocab size
    block_size=128,      # or whatever context size you're training with
    n_layer=6,
    n_head=6,
    n_embd=384,
    dropout=0.1,
    bias=True
)

model = GPT(config)

```

Step 5: Define the loss function

```

def estimate_loss(model):
    out = {}
    model.eval()
    with torch.inference_mode():
        for split in ['train', 'val']:
            losses = torch.zeros(eval_iters)
            for k in range(eval_iters):
                X, Y = get_batch(split)
                with ctx:
                    logits, loss = model(X, Y)
                losses[k] = loss.item()
            out[split] = losses.mean()

```



```
model.train()
return out
```

Step 6: Define SLM Training Configuration Part 1

```
# Training Config
import torch
from contextlib import nullcontext

learning_rate = 1e-4 #more stable training, earlier 1e-4
max_iters = 5000 #increase from 25000
warmup_steps = 1000 #smoother initial train, earlier 100
min_lr = 5e-4 #lower rate, earlier 5e-4
eval_iters = 500 # increased from 100
batch_size = 32 # changed from 16, better gradient estimate
block_size = 128 #changed from 64, capture longer range dependencies

gradient_accumulation_steps = 32 # reduced from 50

device = "cuda" if torch.cuda.is_available() else "cpu"
device_type = 'cuda' if 'cuda' in device else 'cpu' # for later use in torch.autocast
# note: float16 data type will automatically use a GradScaler

# How to use autocast https://wandb.ai/wandb_fc/tips/reports/How-To-Use-Autocast-in-PyTorch--VmlldzoymTK4NTky
dtype = 'bfloat16' if torch.cuda.is_available() and torch.cuda.is_bfloat16_supported() else 'float16' # 'float32', 'bfloat16',
dtype = 'bfloat16' if torch.cuda.is_available() and torch.cuda.is_bfloat16_supported() else 'float16' # 'float32', 'bfloat16',
ptdtype = {'float32': torch.float32, 'bfloat16': torch.bfloat16, 'float16': torch.float16}[dtype]

ctx = nullcontext() if device_type == 'cpu' else torch.amp.autocast(device_type=device_type, dtype=ptdtype)

torch.set_default_device(device)
torch.manual_seed(42)

<torch._C.Generator at 0x78fc01236a90>
```

Step 7: Define SLM Training Configuration Part 2

```
from torch.optim.lr_scheduler import LinearLR, SequentialLR, CosineAnnealingLR

##PUT IN WEIGHT DECAY, CHANGED BETA2 to 0.95
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate, betas=(0.9, 0.95), weight_decay=0.1, eps=1e-9) #weight

scheduler_warmup = LinearLR(optimizer, total_iters = warmup_steps) #Implement linear warmup
scheduler_decay = CosineAnnealingLR(optimizer, T_max = max_iters - warmup_steps, eta_min = min_lr) #Implement lr decay
scheduler = SequentialLR(optimizer, schedulers=[scheduler_warmup, scheduler_decay], milestones=[warmup_steps]) #Switching 1

# https://stackoverflow.com/questions/72534859/is-gradscaler-necessary-with-mixed-precision-training-with-pytorch
scaler = torch.cuda.amp.GradScaler(enabled=(dtype == 'float16'))

/tmp/ipython-input-2132813893.py:11: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp
scaler = torch.cuda.amp.GradScaler(enabled=(dtype == 'float16'))
```

Step 8: Pre-train the SLM

```
best_val_loss = float('inf')
best_model_params_path = "best_model_params.pt"
train_loss_list, validation_loss_list = [], []

# Ensure model is on the correct device
model = model.to(device)

# In your training loop
for epoch in tqdm(range(max_iters)):
    if epoch % eval_iters == 0 and epoch != 0:
        # Ensure estimate_loss uses the correct device
        losses = estimate_loss(model)
        print(f"Epoch {epoch}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")
        print(f"The current learning rate: {optimizer.param_groups[0]['lr']:.5f}")
        train_loss_list += [losses['train']]
        validation_loss_list += [losses['val']]

        if losses['val'] < best_val_loss:
            best_val_loss = losses['val']
            torch.save(model.state_dict(), best_model_params_path)

# Ensure X and y are on the correct device
X, y = get_batch("train")
X, y = X.to(device), y.to(device)
```



```

with ctx:
    logits, loss = model(X, y)
    loss = loss / gradient_accumulation_steps
    scaler.scale(loss).backward()

if ((epoch + 1) % gradient_accumulation_steps == 0) or (epoch + 1 == max_iters):
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=0.5)
    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad(set_to_none=True)
    scheduler.step()

```

100% 5000/5000 [57:03<00:00, 1.91it/s]

Epoch 500: train loss 9.4435, val loss 9.4490
 The current learning rate: 0.00007
 Epoch 1000: train loss 8.4657, val loss 8.4709
 The current learning rate: 0.00010
 Epoch 1500: train loss 7.4796, val loss 7.4778
 The current learning rate: 0.00012
 Epoch 2000: train loss 6.4229, val loss 6.4188
 The current learning rate: 0.00016
 Epoch 2500: train loss 5.4524, val loss 5.4456
 The current learning rate: 0.00022
 Epoch 3000: train loss 4.8235, val loss 4.8173
 The current learning rate: 0.00030
 Epoch 3500: train loss 4.3103, val loss 4.3093
 The current learning rate: 0.00038
 Epoch 4000: train loss 3.9900, val loss 3.9907
 The current learning rate: 0.00044
 Epoch 4500: train loss 3.7717, val loss 3.7805
 The current learning rate: 0.00048

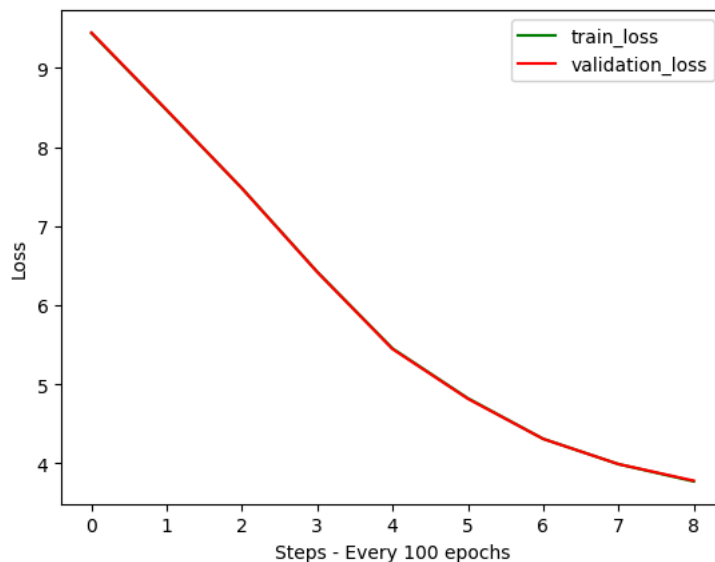
Step 9: Plot the SLM Loss Function

```

import matplotlib.pyplot as plt
train_loss_list_converted = [i.cpu().detach() for i in train_loss_list]
validation_loss_list_converted = [i.cpu().detach() for i in validation_loss_list]

plt.plot(train_loss_list_converted, 'g', label='train_loss')
plt.plot(validation_loss_list_converted, 'r', label='validation_loss')
plt.xlabel("Steps - Every 100 epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

```



Step 10: Run SLM Inference on our trained model

```

#Load the model
model = GPT(config) # re-create the model with same config
device = "cuda" if torch.cuda.is_available() else "cpu"
best_model_params_path = "best_model_params.pt"

if os.path.exists(best_model_params_path):
    model.load_state_dict(torch.load(best_model_params_path, map_location=torch.device(device))) # load best model states
    print(f"Model loaded successfully from {best_model_params_path}")

```

```
else:
    print(f"Error: Model file '{best_model_params_path}' not found. "
          "The training process (Step 8) did not save the model. "
          "This is likely because 'eval_iters' is currently set to 500, which is equal to 'max_iters'. "
          "Please adjust 'eval_iters' in Step 6 (e.g., set it to 50) and re-run Step 8 to ensure the model is saved.")
```

Model loaded successfully from best_model_params.pt

```
sentence = "Once upon a time there was a rabbit."
context = (torch.tensor(enc.encode_ordinary(sentence)).unsqueeze(dim = 0))
y = model.generate(context, 200)
print(enc.decode(y.squeeze().tolist()))
```

Once upon a time there was a rabbit. And, there was a girl named tricks. Once upon it with it to banana little boy who was st
Her mom made higher as she saw a very Whclude. Once upon a time, "You fight that be better. there was a you like to make hers

One under the slide and the park.
As he taken its fun day, came to join the frog. She decided to "
Amy took a whistle andy am so to tell Mom still sad and turned

```
sentence = "A little boy went to the jungle"
context = (torch.tensor(enc.encode_ordinary(sentence)).unsqueeze(dim = 0))
y = model.generate(context, 200)
print(enc.decode(y.squeeze().tolist()))
```

A little boy went to the jungle high and wanted, when her hand. Then she started to eat with a big smile. Suddenly, away and
As they tower with a big very unhappy. After pretty said I decided to be a small water cold to be Her mom and had sure.
But she went excited to delicate one day on the woods before she run across the park with her happy a made anard, but she pl
One had a tall band happily returned to each hat and someone felt two but he thought it lazy truck was a leash and better. C

```
from google.colab import runtime
runtime.unassign()
```